

1. 数据挖掘(Data Mining)实验课II实验报告

0.1. 分析说明

K-Means 是一种经典的无监督学习算法，通过将数据分为 K 个聚类，实现数据的自动分类。在这种场景下，我们要将6张图片分为两个类别。

0.1.1. 图片处理灰度图

加载彩色图像数据并将图像转换为灰度图。

这里使用skimage库的rgb2gray函数，它使用标准的YUV转换公式来实现RGB到灰度的转换，考虑了人眼对不同颜色的敏感度，确保转换后的灰度图像在视觉上更接近人眼看到的效果，从而保留了图像的重要视觉信息。

建议不要使用Pillow库通过 `convert('L')` 将RGB图像转换为灰度图。这种方法在内部可能使用一个简单的平均或者是一个固定的权重比例来计算灰度值，与标准的YUV转换公式略有不同。

处理后将每张图像展平为一维向量。

0.1.2. K-Means算法

K-Means 算法主要分为以下步骤：

1. **初始化中心点**：随机选择 k 个图像作为初始中心点。
2. **分配簇**：计算每张图像与各中心点的欧几里得距离，将其分配到最近的中心。

◦ 欧几里得计算公式为
$$d(x_j, c_i) = \sqrt{\sum_{k=1}^D (x_{jk} - c_{ik})^2}$$

3. **更新中心点**：根据每个簇的成员，计算簇的均值作为新的中心点。
4. **收敛检查**：当中心点的变化小于设定的阈值或达到最大迭代次数时停止迭代。

参数设置：

```
K_CLASS = 2  
  
max_iters = 100  
tolerance = 1e-4
```

0.2. 关键代码

处理图像

```
1 def load_images(image_paths):  
2     images = []  
3     for path in image_paths:  
4         image = io.imread(path)  
5         gray_image = color.rgb2gray(image) # 转换为灰度图  
6         flattened_image = gray_image.flatten() # 展平为一维数组  
7         images.append(flattened_image)  
8     return np.array(images)
```

K-Means算法

```
1 # 随机选择初始中心
```

```

2  def choose_initial_centers(images, K_CLASS):
3      return images[np.random.choice(images.shape[0], K_CLASS, replace=False)]
4
5  # 分配簇
6  def assign_clusters(images, centroids):
7      # 分配每个图像到最近的中心
8      distances = np.sqrt(((images - centroids[:,
9      np.newaxis])**2).sum(axis=2))
10     return np.argmin(distances, axis=0)
11
12 # 更新中心点
13 def update_centroids(images, assignments, K_CLASS):
14     # 计算新的中心点
15     new_centroids = np.array([images[assignments == i].mean(axis=0) for i in
16     range(K_CLASS)])
17     return new_centroids
18
19 # 从加载的图像中随机选择K_CLASS个样本作为初始中心
20 initial_centers = choose_initial_centers(images, K_CLASS)
21 print("Initial centers:", initial_centers)
22
23 # 执行K-Means聚类并可视化中心点的变化
24 centroids = initial_centers
25 centroids_history = [centroids] # 初始化中心点历史记录
26 for iteration in range(max_iters):
27     # 分配图像到最近的中心
28     labels = assign_clusters(images, centroids)
29
30     # 更新中心点
31     new_centroids = update_centroids(images, labels, K_CLASS)
32
33     # 检查中心点是否变化, 如果变化小于tolerance则停止迭代
34     if np.all(np.abs(new_centroids - centroids) < tolerance):
35         print(f"Converged after {iteration + 1} iterations.")
36         break
37     centroids = new_centroids
38     centroids_history.append(centroids) # 更新中心点历史记录

```

整合了上述初始化、分配簇、更新中心点等操作，实现了完整的 K-Means 算法流程。它先初始化中心点，然后在循环中不断重复分配簇和更新中心点的操作，直到满足收敛条件（中心点变化小于给定的容忍阈值或者达到最大迭代次数），同时记录每次迭代后的中心点数据用于后续可视化展示。

0.3. 实验结果:

0.3.1. 处理结果

加载的图像数据为 6 张，展平后的数据形状为：

```

1  Image fig_1.png flattened array shape: (90000,)
2  Image fig_2.png flattened array shape: (90000,)
3  Image fig_3.png flattened array shape: (90000,)
4  Image fig_4.png flattened array shape: (90000,)
5  Image fig_5.png flattened array shape: (90000,)
6  Image fig_6.png flattened array shape: (90000,)

```

其中90000为每张图像展平后的一维长度。

0.3.2. 聚类结果

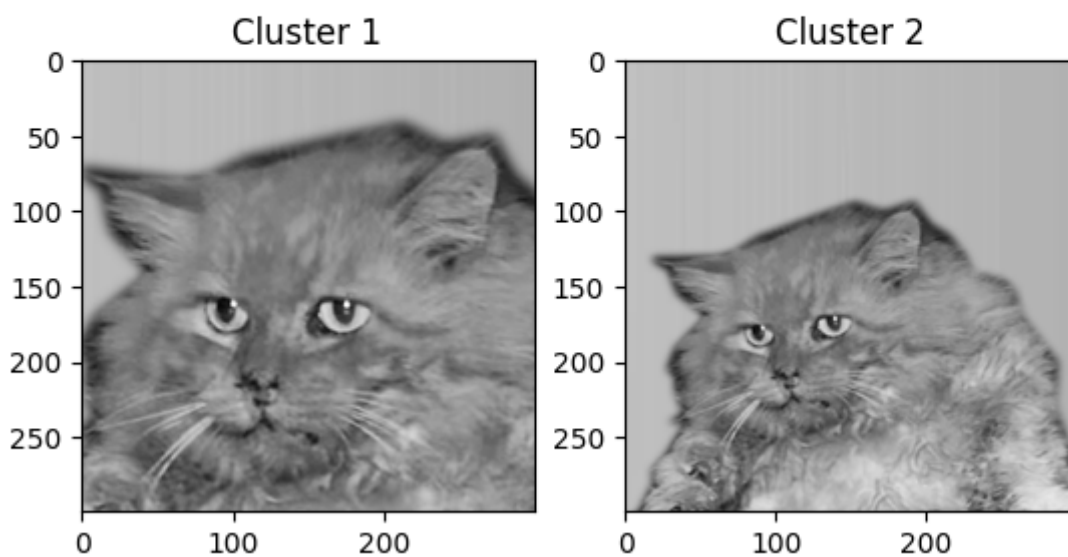
```
1 图片 fig_1.png 被分类为了类别 = 1;  
2 图片 fig_2.png 被分类为了类别 = 2;  
3 图片 fig_3.png 被分类为了类别 = 1;  
4 图片 fig_4.png 被分类为了类别 = 1;  
5 图片 fig_5.png 被分类为了类别 = 2;  
6 图片 fig_6.png 被分类为了类别 = 2;
```

成功地将6张图像分为两类，并且快速收敛。

0.3.3. 收敛情况

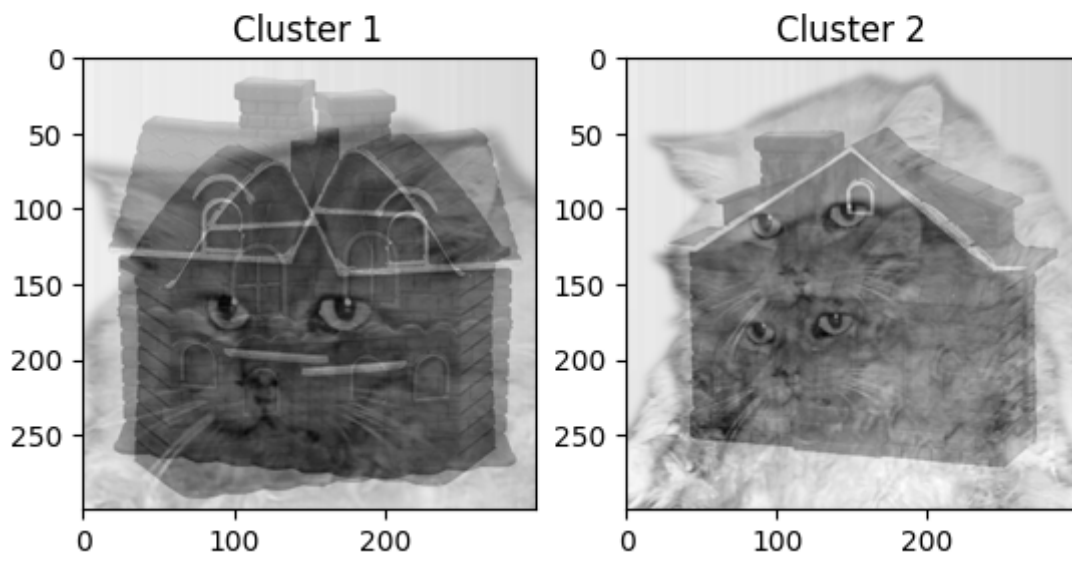
算法在第4次迭代后收敛（实际运行的迭代次数会因初始中心点不同而有所变化）

Iteration 1

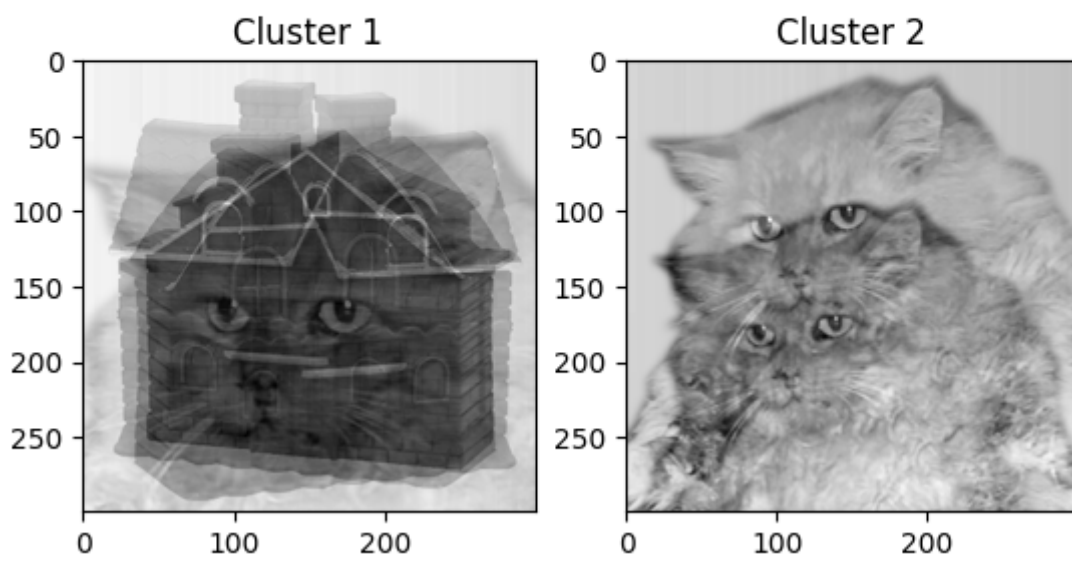


随机选择初始中心点

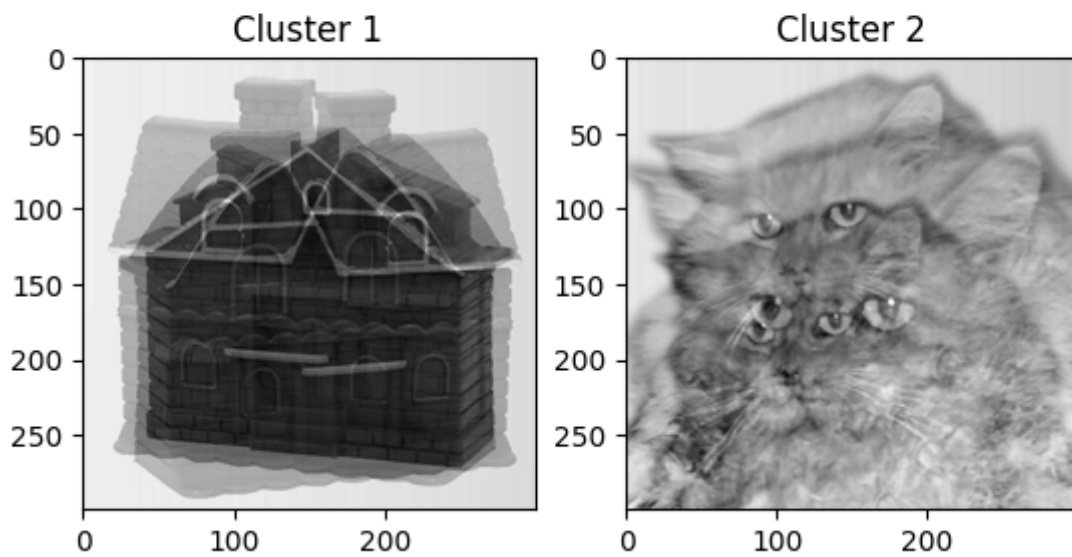
Iteration 2



Iteration 3



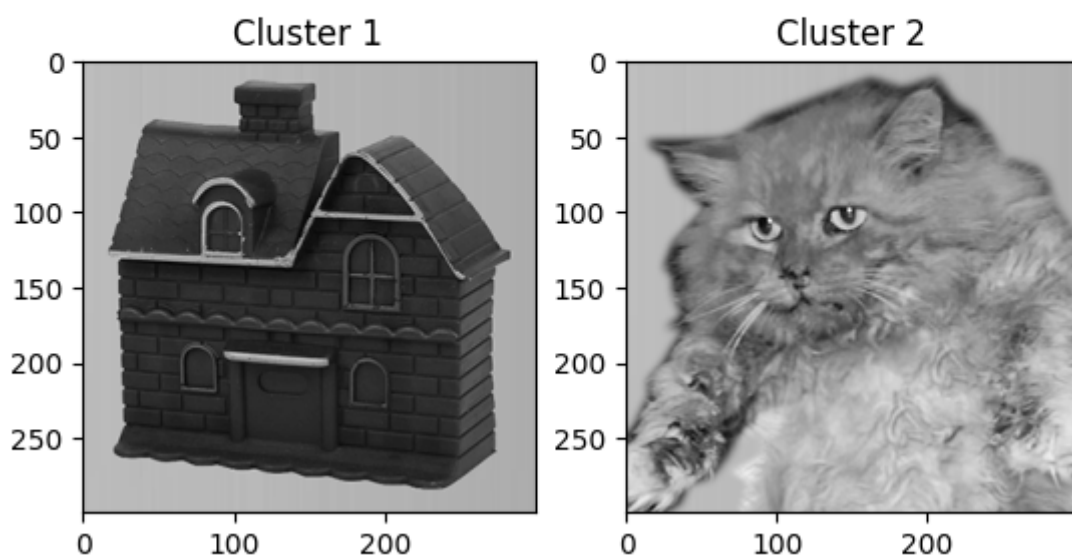
Iteration 4



从这些迭代过程中可以看出，算法在逐步调整聚类中心，使每个聚类更好地代表其分配的数据点。聚类最后将图像分成了两类：一类是房子，另一类是猫

由于初始化中心是随机的,结果可能在不同的运行之间有所不同;观察多次运行的不同结果，以下是不同结果：

Iteration 1



Iteration 2

